

HelixTrack

HelixTrack

JIRA-Alternative für die freie Welt.

Zusammenfassung

HelixTrack ist eine umfassende, moderne Open-Source-Alternative zu JIRA (und über die Documents-Erweiterung auch zu Confluence) – ein plattformübergreifendes Projektmanagement- und Issue-Tracking-System, das auf einem Go-Microservices-Backend basiert und native Clients für Web, Desktop und Mobile bietet.

Kurzbeschreibung

Open-Source-Alternative zu JIRA/Confluence. Ein Go-Microservices-Backend („HelixTrack Core“) stellt eine einheitliche REST-API-Schnittstelle für Projekt- und Issue-Tracking sowie einen Confluence-ähnlichen Dokumentenarbeitsbereich bereit, die über HTTP/3-QUIC an native Web-, Desktop-, Android- und iOS-Clients ausgeliefert wird.

Ausführliche Beschreibung

HelixTrack ist eine Open-Source-Plattform für Projektmanagement und Issue-Tracking, die als Alternative aus der freien Welt zu JIRA und Confluence positioniert ist – ein vollständiger Ersatz für die beiden Produkte, in die die meisten technischen Organisationen fest eingebunden sind, neu entwickelt als Software, die Ihnen gehört und die Sie überall betreiben können. Ihr Herzstück ist **HelixTrack Core**, ein REST-API-Microservice in Go, entwickelt mit dem Gin-Framework. Er bietet vollständiges Issue-Tracking, Agile/Scrum-Boards, Teamverwaltung und eine hierarchische Berechtigungsengine, deren Implementierung zwischen einer lokalen In-Process-Engine und einem HTTP-basierten Dienst austauschbar ist – sodass

dasselbe Autorisierungsmodell von einem einzelnen Laptop bis hin zu einem verteilten Cluster skaliert, ohne den Anwendungscode anzupassen. Statt eine REST-Oberfläche über Dutzende Routen zu verteilen, leitet Core alles über einen einzigen, aktionsbasierten / do-Endpunkt mit einem einheitlichen Anfrage-/Antwort-Schema (action/jwt/object/data rein, errorCode/errorMessage/data raus): Jeder Client spricht denselben minimalen Vertrag, und das Hinzufügen einer Funktion bedeutet das Hinzufügen einer Aktion, nicht eines neuen URL, der dokumentiert, abgesichert und versioniert werden muss. Core integriert sich mit entkoppelten Authentifizierungs-, Berechtigungs- und Lokalisierungsdiensten, die über HTTP/3-QUIC kommunizieren und auf separaten Maschinen oder Clustern laufen können – oder in Testkonfigurationen vollständig deaktiviert werden. Die Daten werden in SQLite für die Entwicklung ohne Setup-Aufwand und in PostgreSQL für die Produktion gespeichert und standardmäßig mit SQLCipher (AES-256) verschlüsselt, sodass sensible Projektdaten auf der Festplatte geschützt sind – nicht als nachträglicher Gedanke, sondern von vornherein. Die **Documents V2**-Erweiterung verwandelt den Tracker in eine vollständige Wissensplattform: ein Confluence-ähnlicher Arbeitsbereich mit Bereichen, Seiten, Versionskontrolle, Vorlagen, Echtzeit-WebSocket-Kollaboration und Analysen – endlich Wiki und Issue-Tracker in einem Backend vereint, statt zwei Produkte künstlich zusammenzufügen. Um Core herum gruppieren sich mehrere Client-Anwendungen: ein Angular-Webclient, ein Tauri- + Angular-Desktopclient, native Android- (Kotlin) und iOS-Apps (Swift) sowie Clients für HarmonyOS und Aurora OS und ein Bildschirmschoner – alle kommunizieren mit demselben Backend und finden es automatisch im lokalen Netzwerk über UDP-Broadcast, sodass ein neuer Client seinen Server ohne manuelle Konfiguration erkennt. Die Client-Anwendungen werden in separaten, privaten Repositories gepflegt und sind hier nur auf Produktebene dargestellt.

Warum wir es entwickelt haben

Um Teams eine wirklich offene, selbst hostbare Alternative zum JIRA- und Confluence-Stack zu bieten – „für die freie Welt“ – ohne Vendor-Lock-in, die unternehmensreife Tracking-, Dokumenten- und Kollaborationsfunktionen unter einer einzigen Open-Source-Lizenz vereint.

Warum es ein Game-Changer ist

Es vereint zwei schwere kommerzielle Produkte – Issue-Tracking und das Wiki-/Dokumenten-Stack – in einer offenen, leistungsstarken, selbst hostbaren Plattform und kombiniert dies mit etwas, das die etablierten Anbieter nie boten: echte *native*

Multiplattform-Clients (Web, Desktop, Android, iOS sowie HarmonyOS und Aurora OS), die alle von einem einzigen Backend-Vertrag gesteuert werden. Der entscheidende Vorteil ist Eigentümerschaft ohne Kompromisse. Ein HTTP/3-überall-Ansatz, ein vollständig entkoppeltes Microservices-Design und SQLCipher-AES-256-Verschlüsselung im Ruhezustand bringen die Leistung und Sicherheit, die sonst nur proprietären SaaS-Lösungen vorbehalten ist, in ein System, das Sie selbst hosten – ohne Benutzerlizenzen, ohne Vendor-Lock-in, ohne dass Daten Ihre Infrastruktur verlassen. Teams erhalten das gewohnte JIRA-plus-Confluence-Erlebnis auf ihrer eigenen Hardware und unter einer einzigen Open-Source-Lizenz.

Was innovativ ist

- Vereinheitlichte aktionsbasierte /do-API – ein Endpunkt, ein Umschlag, aktionsgesteuerte Weiterleitung. Neue Funktionen kommen als neue Aktionen, nicht als neue URLs, was die Angriffsfläche, den Client-Code und den Dokumentationsaufwand auf einen einzigen, von allen Plattformen geteilten Vertrag reduziert.
- HTTP/3-QUIC als *Standard*-Transport zwischen Diensten – moderne, latenzarme und verbindungsresiliente Vernetzung zwischen Services von Anfang an, nicht nachträglich aufgesetzt.
- Ein Berechtigungsmodul, das zwischen einer lokalen In-Process-Implementierung und einem HTTP-basierten Dienst austauschbar ist, kombiniert mit optionalen, unabhängig einsetzbaren Authentifizierungs-, Berechtigungs- und Lokalisierungsdiensten – dasselbe Autorisierungsmodell, egal ob Sie einen einzelnen Prozess oder ein Cluster betreiben.
- Datenisolation über mehrere Bereiche hinweg mittels --space-root-Flag: Jedes Projekt erhält eine eigene isolierte Datenbank und einen eigenen Asset-Speicher, sodass Mandanten und Projekte bereits auf Speicherebene getrennt werden, statt durch Abfragefilter.
- SQLCipher-AES-256-Verschlüsselung im Ruhezustand – sensible Projektdaten werden standardmäßig transparent auf der Festplatte geschützt.
- Automatische Client-zu-Server-Erkennung via UDP-Broadcast im lokalen Netzwerk – ein Client findet Core ohne manuelle Konfiguration.
- Dokumente V2, eine echte „Confluence-Alternative“, mit optimistischem Sperren für paralleles Bearbeiten, Konflikt-Erkennung und vollständiger Versionshistorie – echte kollaborative Dokumente, die dasselbe Backend nutzen wie der Tracker.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Sechs Client-Plattformen, ein Backend, null Vertragsabweichung.** Die Wartung von Web/Angular-, Desktop/Tauri-, Android/Kotlin-, iOS/Swift-, HarmonyOS- und Aurora-Clients bedeutet normalerweise sechs divergierende API-Integrationen, die mit der Zeit auseinanderlaufen. Wir haben dieses Risiko minimiert, indem wir den einzigen aktionsgesteuerten /do-API-Endpunkt und dessen festes Umschlagformat zum *einzigsten* Vertrag gemacht haben – jeder Client richtet sich identisch danach aus. Darüber hinaus haben wir UDP-Broadcast-basierte Diensterkennung integriert, sodass Clients Core im Netzwerk ohne manuell konfigurierte Endpunkte finden.
- **Entkopplung von Diensten ohne Latenz-Nachteil.** Die Aufteilung von Authentifizierung, Berechtigungen und Lokalisierung in unabhängig einsetzbare Dienste führt normalerweise zu einem zusätzlichen Netzwerk-Hop pro Aufruf. Wir haben HTTP/3-QUIC für alle Dienst-zu-Dienst-Kommunikation eingeführt, um diese Hops schnell und verbindungsresilient zu halten, und jeden Dienst so gestaltet, dass er unabhängig ausgeführt werden kann – sogar vollständig deaktiviert in Testkonfigurationen. Die Entkopplung ist somit eine Bereitstellungsoption, kein fester Kostenfaktor.
- **Confluence-reife Kollaboration ohne verlorene Änderungen.** Echtzeit-Bearbeitung durch mehrere Autoren birgt das Risiko von Schreibkonflikten. Dokumente V2 löst dies mit Bereichen/Seiten/Versionierung unter optimistischem Sperren, expliziter Konflikterkennung, einer vollständigen Änderungshistorie als Rückfalloption und Echtzeit-WebSocket-Synchronisation – Kollaboration, die konsistent bleibt, statt Änderungen stillschweigend zu überschreiben.
- **Verschlüsselung im Ruhezustand ohne Durchsatzseinbußen.** SQLCipher-AES-256 schützt Daten auf der Festplatte, verursacht aber pro Abfrage Overhead. Wir haben dies durch mehrschichtiges Caching ausgeglichen (In-Memory-LRU vor Redis im Lokalisierungsdienst), sodass häufig genutzte Pfade wie mehrsprachige Abfragen schnell bleiben, während die Daten verschlüsselt bleiben.

Tech-Stack

- **Go + Gin** – ausgewählt für Hochdurchsatz- und Niedriglatenz-HTTP-Dienste mit einer Single-Binary-Bereitstellung; beinhaltet Core's REST-API, die JWT/CORS-Middleware sowie den aktionsbasierten /do-Router, der das gesamte System bedient.

- **HTTP/3 QUIC** – als Transportschicht zwischen Core und den Authentifizierungs-, Berechtigungs- und Lokalisierungsdiensten gewählt, da QUICs multiplexes, verbindungs-migreres Design Tail-Latenzen reduziert und instabile Verbindungen übersteht, bei denen TCP ins Stocken gerät.
- **PostgreSQL (Prod) / SQLite (Dev)** – ein relationales Modell, das das umfangreiche Tracking- und Dokumentenschema für beide Engines unterstützt: SQLite ermöglicht eine lokale Entwicklung ohne Setup und basiert auf Dateien, während PostgreSQL in der Produktion über ein dediziertes production-Compose-Profil skaliert.
- **SQLCipher (AES-256)** – für transparente, datenbankseitige Verschlüsselung im Ruhezustand gewählt, sodass der Schutz sensibler Projektdaten keine kryptografischen Anpassungen auf Anwendungsebene erfordert und Abfragen unverändert bleiben.
- **Redis** – als gemeinsame Caching-Ebene hinter einem In-Memory-LRU-Cache im Lokalisierungsdienst eingesetzt, um zweistufiges Caching zu ermöglichen, das häufige mehrsprachige Abfragen auch bei Verschlüsselungs-Overhead schnell hält.
- **Uber Zap + Lumberjack** – für strukturiertes, speichereffizientes Logging mit integrierter Rotation gewählt, damit Core in der Produktion beobachtbar bleibt, ohne dass die Logs unkontrolliert wachsen.
- **golang-jwt / JWT** – als zustandsloses Authentifizierungsverfahren gewählt; das signierte Token wird im jwt-Feld jeder /do-Nachricht mitgeführt, sodass die Authentifizierung für alle Clients einheitlich ist.
- **Angular 19 (+ Material, RxJS)** – für eine reaktive, komponentenbasierte Browseranwendung mit einem ausgereiften Material-Design-System out of the box gewählt.
- **Tauri 2.0 + Rust + Angular** – um eine native Desktop-Shell mit minimalem Footprint bereitzustellen, indem die Angular-UI in einem Rust-basierten Webview wiederverwendet wird, statt einen vollständigen Browser-Runtime zu bündeln.
- **Kotlin (Android) / Swift + SwiftUI (iOS)** – gewählt, damit mobile Nutzer echte native, plattformtypische Clients erhalten, statt einer eingepackten Webansicht.
- **Docker / Docker Compose (Podman-kompatibel)** – für reproduzierbare, containerisierte Bereitstellung mit integrierten /health-Checks gewählt, sowie Podman-Kompatibilität, sodass kein Daemon oder Anbieter zwingend erforderlich ist.
- **Testify (Go); Cypress/Playwright/Karma+Jasmine (Clients)** – für mehrschichtiges automatisiertes Testen gewählt, das den Backend-Vertrag und die Client-UIs unabhängig abdeckt und zur Architektur mit einem Backend und vielen Clients passt.

Status & Transparenzhinweise

- **Status: Beta.** HelixTrack Core ist ein funktionierender REST-API-Mikrodienst; die **Dokumente-V2**-Erweiterung ist zu etwa 95 % fertiggestellt, weist jedoch ein bekanntes Problem bei der Datenbankfeldzuordnung auf und gilt daher nicht als vollständig ausgeliefert.
- **Lizenz: Noch nicht festgelegt.** Die CLAUDE.md gibt MIT an, während die maßgebliche core/LICENSE-Datei Apache 2.0 enthält – diese Diskrepanz muss vor einer endgültigen Lizenzangabe geklärt werden.
- Die im README des Projekts genannten Leistungsangaben (z. B. 50.000+ Anfragen/Sekunde, Abfragezeiten unter einer Millisekunde) sind geplante Design- und Marketingziele, keine unabhängig veröffentlichten Benchmarks, und werden daher hier nicht als Fakten aufgeführt.
- Die Client-Anwendungen (Web, Desktop, Android, iOS, Aurora, HarmonyOS) befinden sich in **privaten** Repositories und werden nur auf Produktebene beschrieben.

Prioritätsstufe: Helix-primary und das Flaggschiff der Helix-Track-Produktlinie – rangiert vor allen Server Factory-Projekten.